



Foreach_Loops

🕒 Created time	September 4, 2026 11:05 AM
☰ Category	Loops_Sets
👤 Last edited by	 R.A R.A
🕒 Last updated time	September 4, 2026 11:09 AM

Foreach Loop

```
string data = "Tim, Sue, Bob, Jane";  
  
List<string> firstNames = data.Split(',').ToList();  
  
foreach (string firstName in firstNames)  
{  
    Console.WriteLine(firstName);  
}
```

לולאת **foreach**

דרך פשוטה לבצע מעבר בלולאה על אוסף של פריטים.

המשמעות היא לעבור על כל אלמנט באוסף.

אם הרשימה גדלה ונוספים אליה פריטים, הלולאה תעבור גם על יותר פריטים.

אם הרשימה קטנה, הלולאה תעבור על פחות פריטים.

ואם הרשימה ריקה, הלולאה בכלל לא תבצע את הקוד שבתוכה.

יש לנו רשימה של שמות פרטיים.

שימו לב שאנחנו משתמשים כאן בשם ברבים:

```
foreach (string firstName in firstNames)
{

}
```

`firstNames` הוא ברבים.

השימוש בשם ברבים חשוב, מפני שהוא אומר לנו שמדובר ב־אוסף (Set).

זו אחת הסיבות לכך ששמות של משתנים הם חשובים.

כאשר יש לנו משתנה עם שם ברבים, אנחנו מצפים שיהיה בו יותר מאלמנט אחד.

זה יכול להיות למשל:

- `Array`
- `List`
- `Dictionary`

כלומר, סוג כלשהו של אוסף של נתונים.

לעומת זאת, כאן:

```
foreach (string firstName in firstNames)
{

}
```

`firstName` הוא ביחיד.

המשמעות היא שהוא מחזיק פריט אחד בכל פעם.

במקרה הזה, לולאת `foreach` בעצם אומרת:

עבור כל שם פרטי שנמצא בתוך אוסף השמות `firstNames`, אנחנו רוצים ליצור משתנה מסוג `string` בשם `firstName`.

כלומר:

עבור כל `firstName` שנמצא בתוך `firstNames`.

הלולאה תשתמש במשתנה הזה עבור כל אלמנט שנמצא ברשימה שלנו.

האם אכפת לנו כמה פריטים יש ברשימה?

לא.

ה- `foreach` מטפל בזה בשבילנו.

האם אנחנו יודעים באיזה סדר הוא שולף את הפריטים?
כן.

הוא שולף אותם לפי הסדר שבו הם שמורים ברשימה.

אבל אנחנו לא מקבלים את `i`, כלומר לא מקבלים את האיטרטור / המונה או את מספר המיקום של הפריט במערך.

במקום זה, מה שאנחנו מקבלים הוא המשתנה שנקרא:

`firstName`

```
foreach (string firstName in firstNames)
{
    Console.WriteLine(firstName);
}
```

כאשר נריץ את הקוד הזה, נראה:

```
Tim
Sue
Bob
Jane
```

האפשרות להשתמש ב- `var`

אפשר גם לראות את אותה לולאה כתובה כך:

```
foreach (var firstName in firstNames)
{
    Console.WriteLine(firstName);
}
```

יכול להיות שנראה שימוש ב- `var`.

`var` הוא פשוט קיצור שמאפשר לנו לא לכתוב במפורש את שם הטיפוס האמיתי.

`var` "variable" לא אומר.

המשמעות שלו היא שהטיפוס של המשתנה יהיה הטיפוס שנקבע עבורו כאשר הוא קיבל את הערך שלו בפעם הראשונה.

